Nama: Muh. Farraz Nawwaf

NIM: 60900124050

Kelas: B

A. Setup, Pembuatan Tabel & Seed Data

Langkah 1: Buat Database dan dan Ownernya

```
-- 1. Buat user dan database baru sesuai standar modul
CREATE USER skillbridge WITH PASSWORD 'secret';
CREATE DATABASE skillbridge OWNER skillbridge;

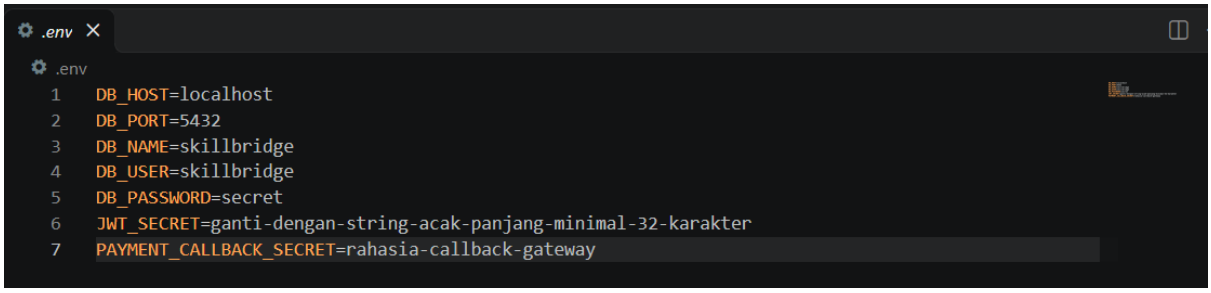
-- 2. Pastikan user skillbridge punya hak penuh di schema public
\c skillbridge
GRANT ALL ON SCHEMA public TO skillbridge;
\q
```

Screenshot:

```
postgres=# -- 2. Buat user dan database baru sesuai standar modul
postgres=# CREATE USER skillbridge WITH PASSWORD 'secret';
CREATE ROLE
postgres=# CREATE DATABASE skillbridge OWNER skillbridge;
CREATE DATABASE
postgres=#
postgres=# -- 3. Pastikan user skillbridge punya hak penuh di schema public (khusus PostgreSQL 15/16+)
postgres=# \c skillbridge
You are now connected to database "skillbridge" as user "postgres".
skillbridge=# GRANT ALL ON SCHEMA public TO skillbridge;
GRANT
skillbridge=# \q
```

Langkah 2: Pastikan File .env Sesuai Bawaan

Pastikan file env sama dengan pembuatan database terutama DB_NAME, DB_USER, dan DB_PASSWORD



Langkah 3: Buat Tabel, Seed Data, dan Nyalakan Server

Di terminal VS Code (pastikan .venv sudah aktif), jalankan perintah ini secara berurutan:

```
# 1. Buat seluruh tabel di database skillbridge
python -m app.create_tables

# 2. Masukkan data awal (admin, instructor, kategori, kursus, 4 modul)
python -m app.seed

# 3. Jalankan server FastAPI
uvicorn app.main:app --reload
```

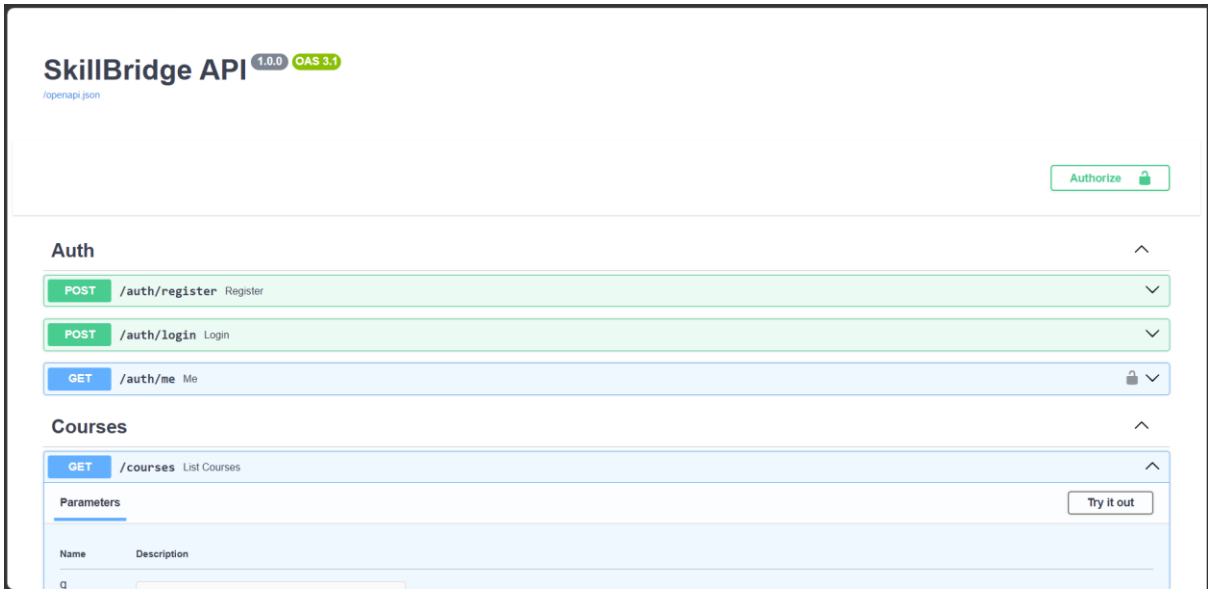
Screenshot:

```
(.venv) PS D:\VSCode\Pemrogramanlanjut\skillbridge-api> python -m app.create_tables
Tabel berhasil dibuat.
(.venv) PS D:\VSCode\Pemrogramanlanjut\skillbridge-api>
```

```
(.venv) PS D:\VSCode\Pemrogramanlanjut\skillbridge-api> python -m app.seed
Seed berhasil.
(.venv) PS D:\VSCode\Pemrogramanlanjut\skillbridge-api>
```

```
(.venv) PS D:\VSCode\Pemrogramanlanjut\skillbridge-api> uvicorn app.main:app --reload
INFO:     Will watch for changes in these directories: ['D:\\VSCode\\Pemrogramanlanjut\\skillbridge-api']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [22120] using WatchFiles
INFO:     Started server process [20208]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

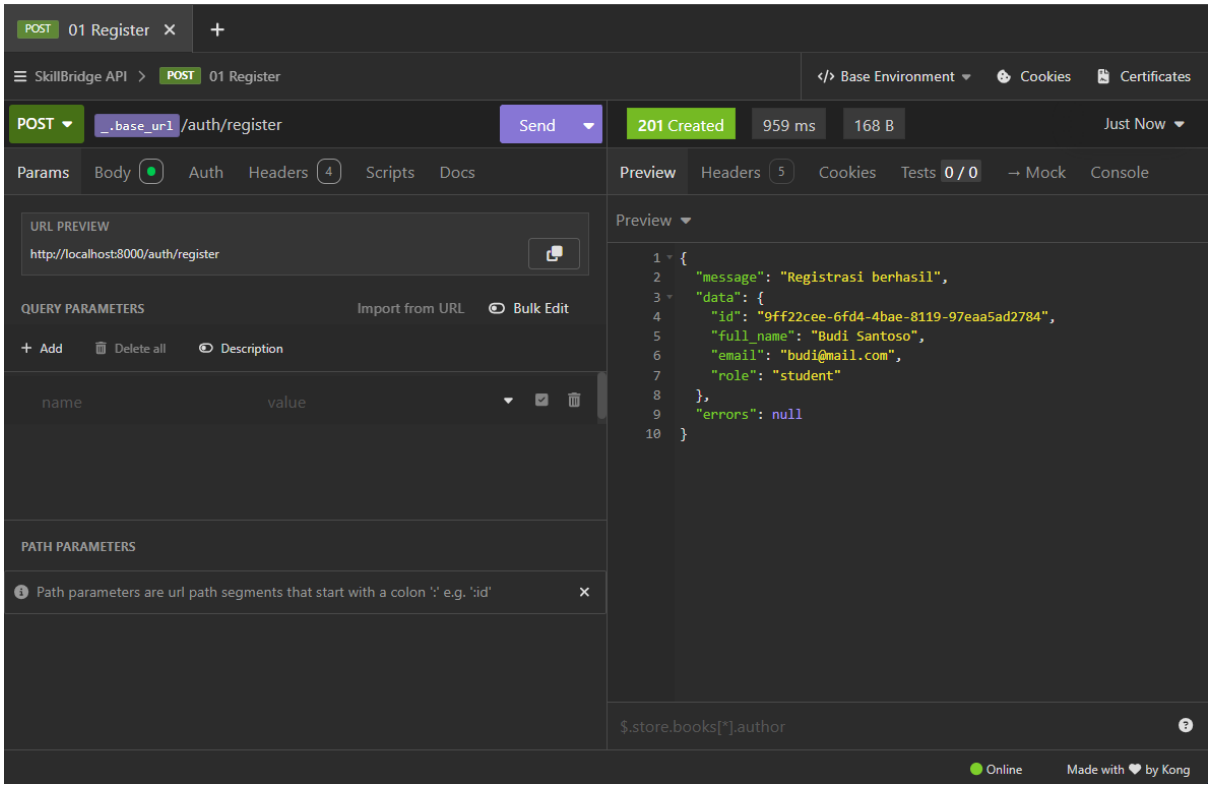
Bukti server sudah berjalan:



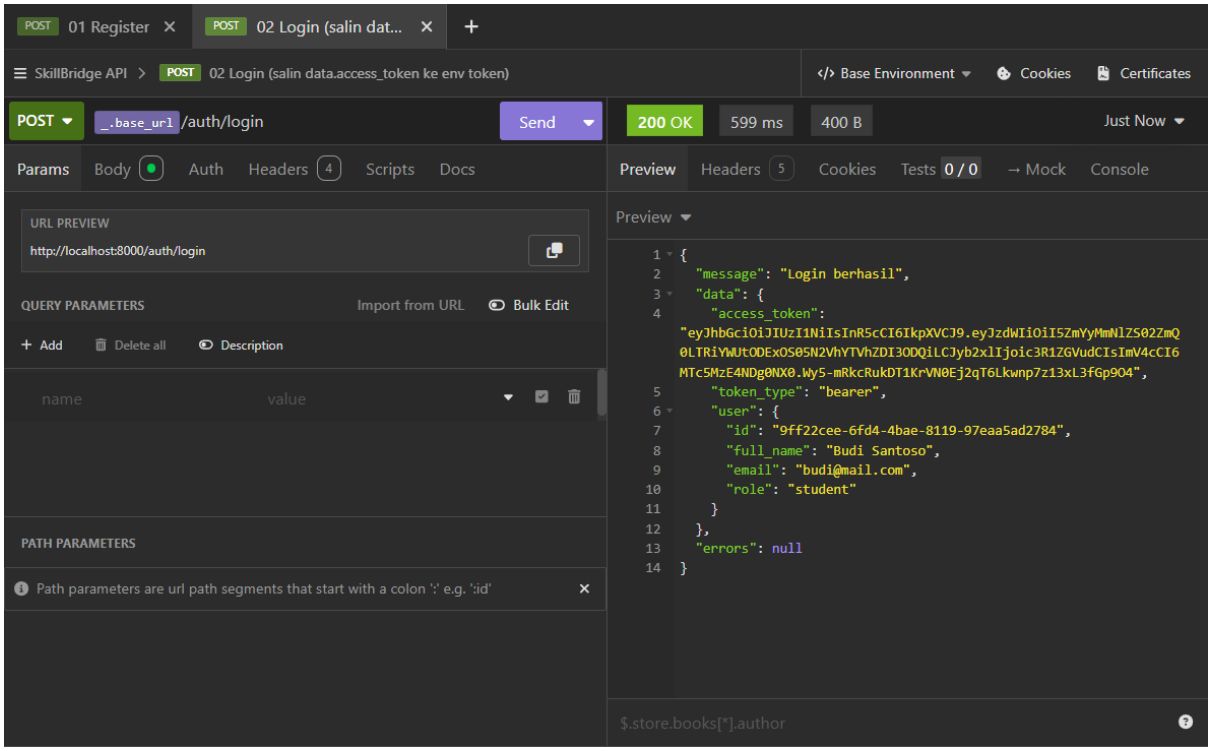
Setelah itu lakukan import file SkillBridge.insomnia.json ke dalam Aplikasi Insomnia untuk melakukan pengujian

B. Pengujian Alur Positif di Insomnia

Pengujian 1: Daftar Akun Student Baru



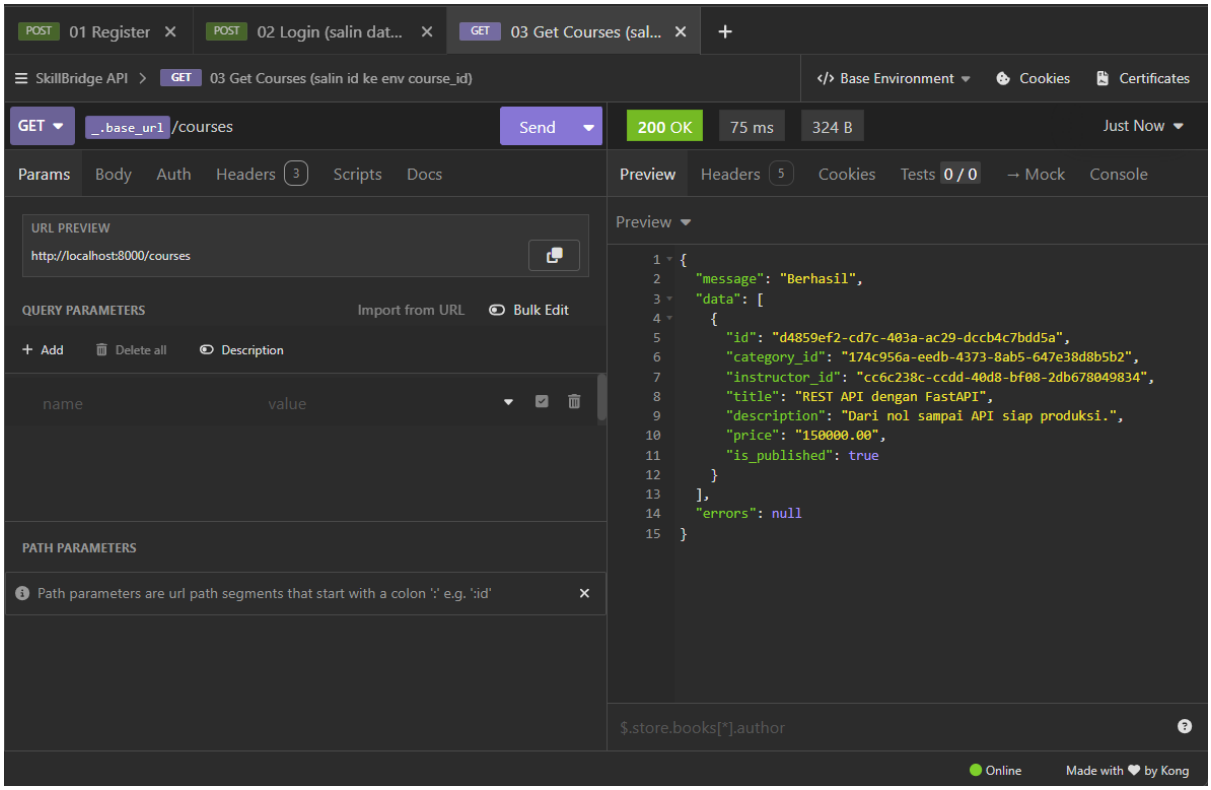
Pengujian 2: Login Student & Ambil Token JWT



Langkah tambahan:

Blok dan *copy* isi string dari "access_token" (tanpa tanda kutip). Buka **Manage Environments**, tempelkan (*paste*) ke dalam "token": "TEMPEL_DI_SINI", lalu tutup jendela Environment.

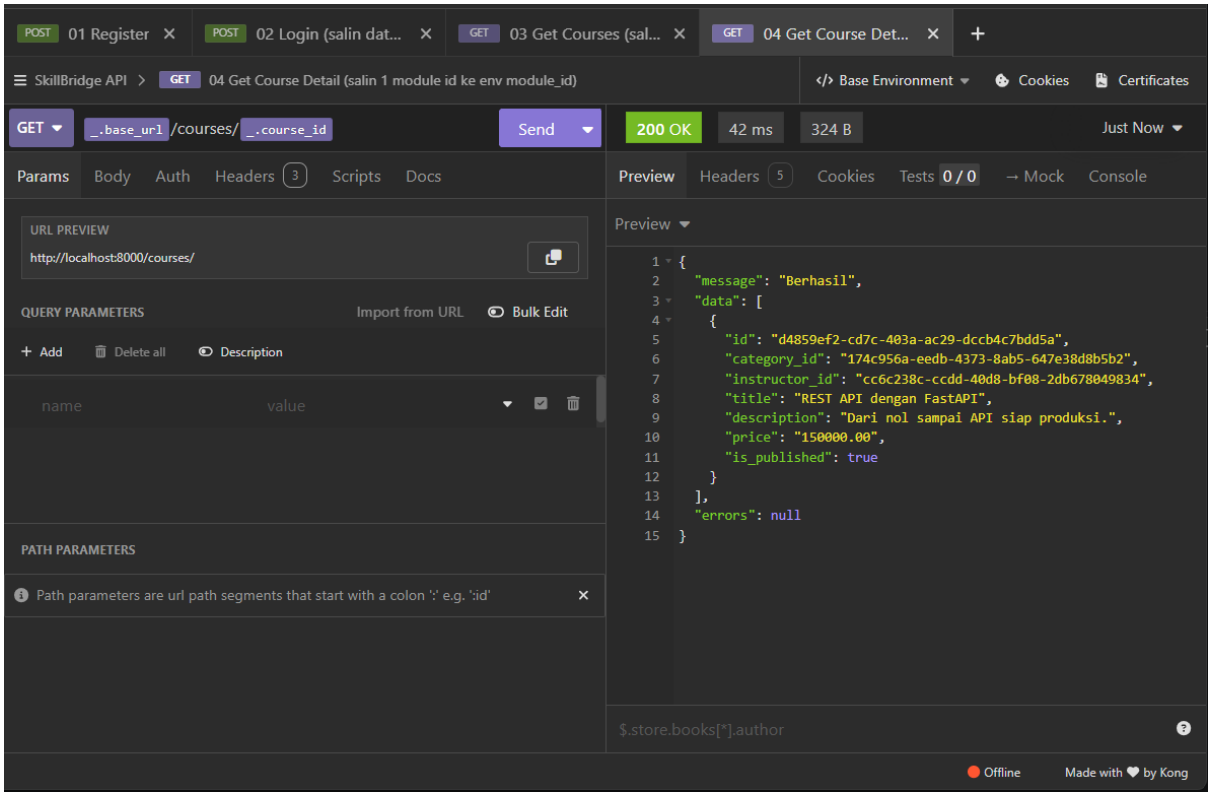
Pengujian 3: Login Student & Ambil Token JWT



Langkah tambahan:

Perhatikan di dalam "data": [{ ... }], ada 3 UUID (id, category_id, instructor_id). **Jangan salah pilih:** salin nilai milik **"id"** (baris paling atas di dalam objek kursus). Buka **Manage Environments**, tempelkan ke dalam "course_id": "TEMPEL_DI_SINI".

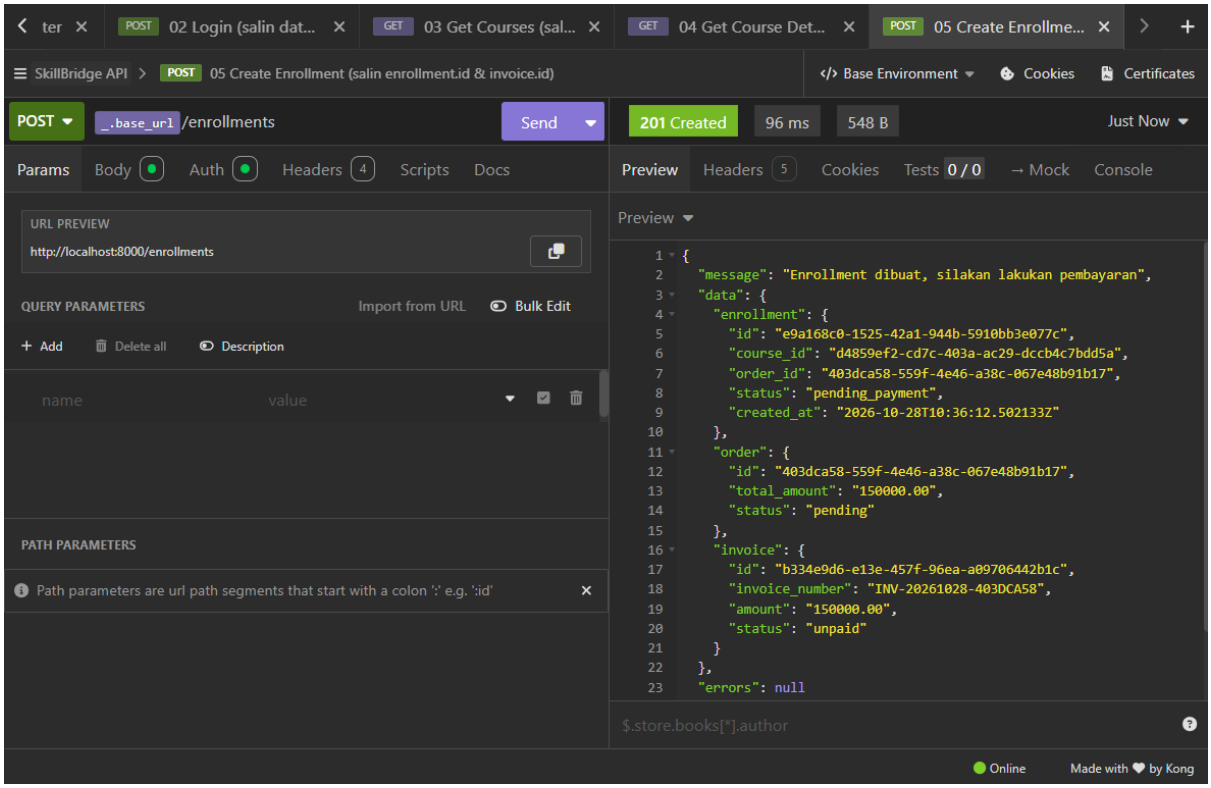
Pengujian 4: Lihat Detail Kursus & Daftar Modul



Langkah tambahan:

Lihat ke dalam array "modules", salin nilai **"id" dari modul pertama** ("position": 1, "title": "Pengenalan REST"). Buka **Manage Environments**, tempelkan ke dalam "module_id": "TEMPEL_DI_SINI".

Pengujian 5: Daftar ke Kursus / Buat Enrollment



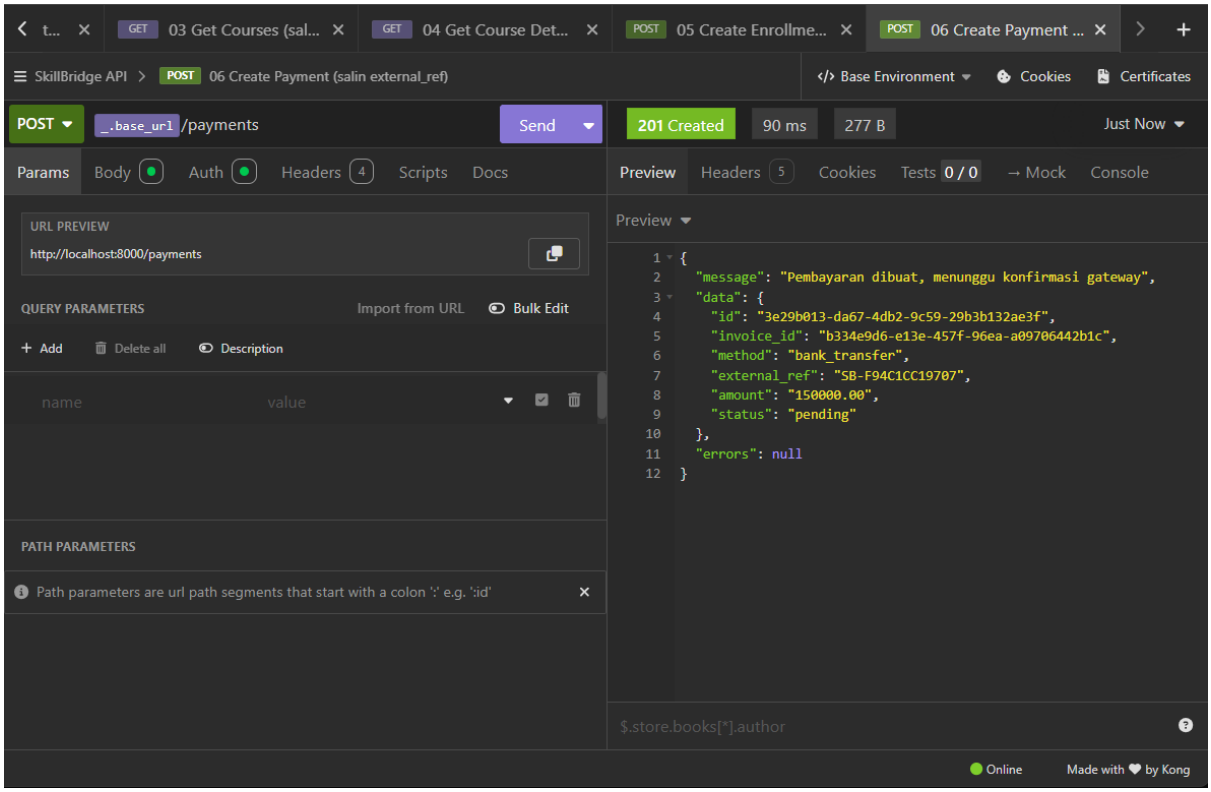
Catatan:

Status harus **201 Created**. (Jika muncul 401, berarti token dari langkah #2 belum ditempel ke Environment). Perhatikan di Response muncul 3 objek sekaligus: "enrollment" (status: "pending_payment"), "order" (status: "pending"), dan "invoice" (status: "unpaid").

Langkah tambahan:

Salin "id" yang ada di dalam blok **"enrollment"** → tempelkan ke "enrollment_id" di **Manage Environments**. Salin "id" yang ada di dalam blok **"invoice"** → tempelkan ke "invoice_id" di **Manage Environments**. (Hati-hati: jangan menyalin id milik order, pastikan yang disalin adalah enrollment.id dan invoice.id).

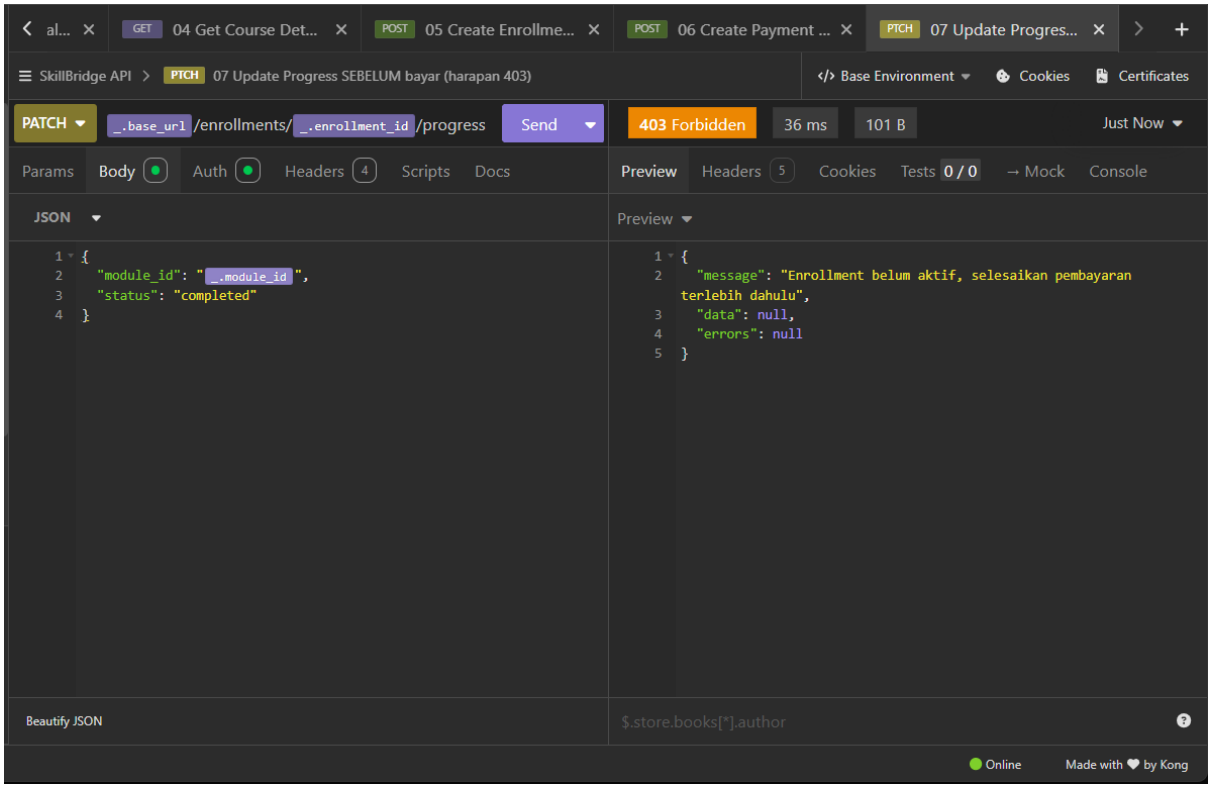
Pengujian 6: Buat Instruksi Pembayaran



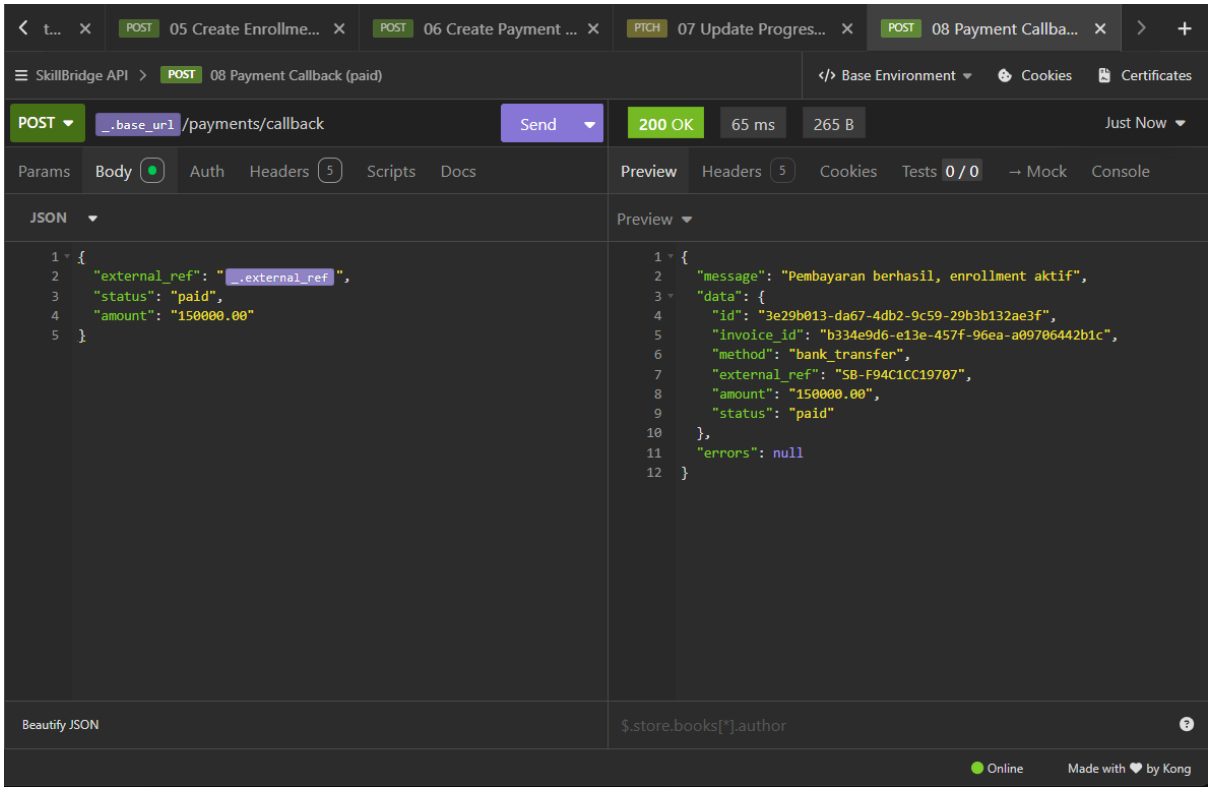
Langkah tambahan:

Salin nilai dari "external_ref" (tanpa tanda kutip). Buka **Manage Environments**, tempelkan ke dalam "external_ref": "TEMPEL_DI_SINI".

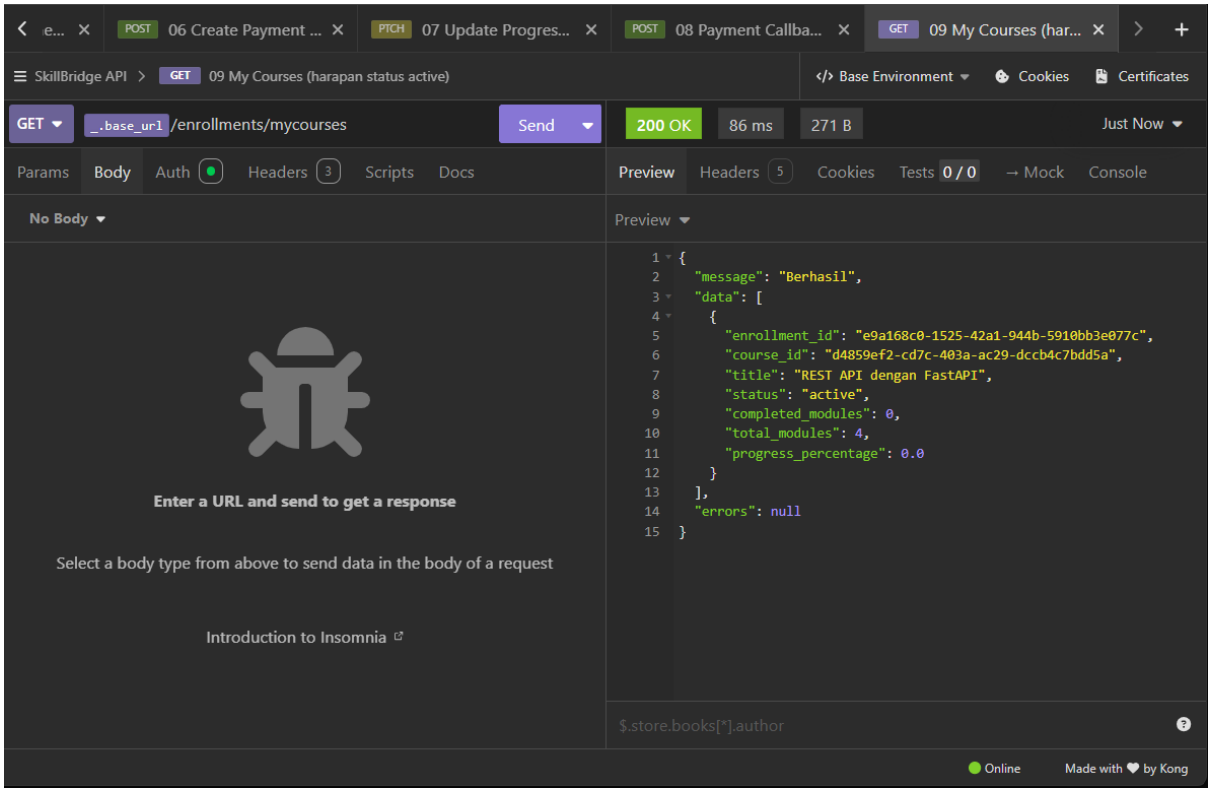
Pengujian 7: Uji Coba Update Progress SEBELUM Bayar



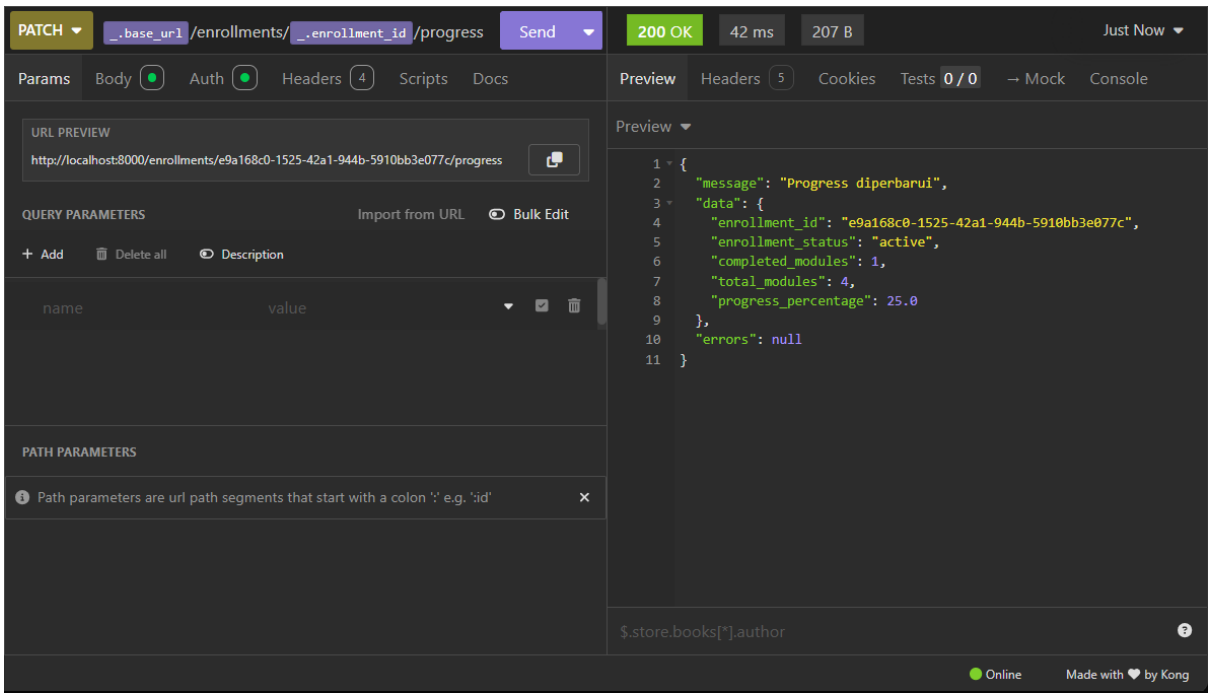
Pengujian 8: Simulasi Callback Pembayaran Berhasil



Pengujian 9: Cek Kursus Saya



Pengujian 10: Update Progress SETELAH Bayar



Tabel Ringkasan:

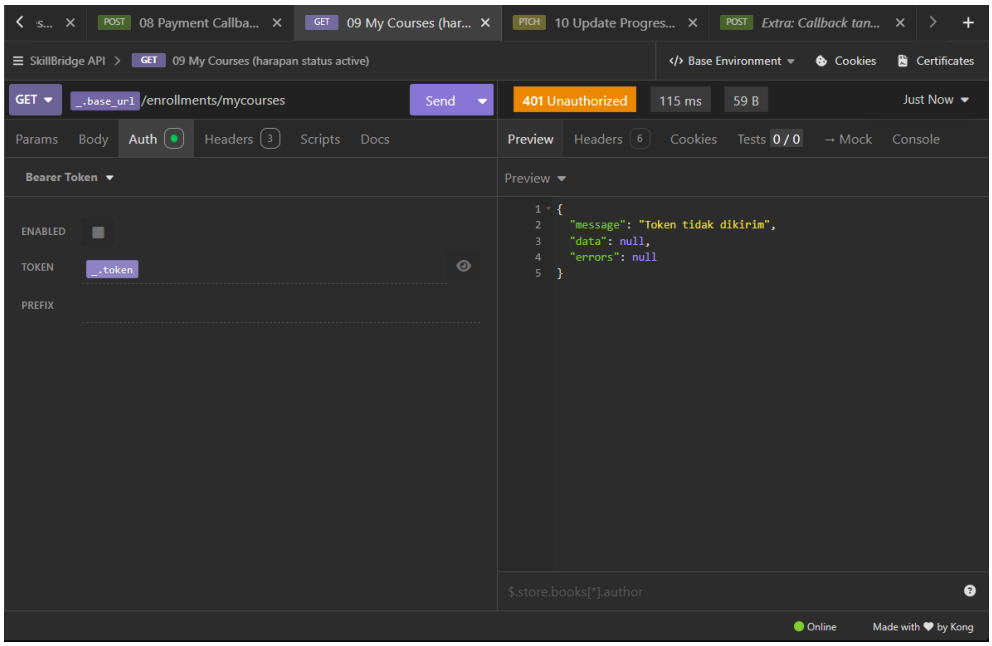
No	Skenario	Hasil Diharapkan	Hasil Aktual	Keterangan
1.	Register	201	201	Berhasil
2.	Login	200	200	Berhasil
3.	Get Courses	200	200	Berhasil
4.	Get Course Detail	200	200	Berhasil
5.	Create Enrollment	201	201	Berhasil
6.	Create Payment	201	201	Berhasil
7.	Update Progress Sebelum Bayar	403	403	Berhasil
8.	Payment Callback (Paid)	200	200	Berhasil
9.	My Courses	200	200	Berhasil
10.	Update Progress Setelah Bayar	200	200	Berhasil

B. Pengujian Alur Negatif di Insomnia

Pengujian 1: Tanpa Token (401 Unauthorized)

Langkah pengujian:

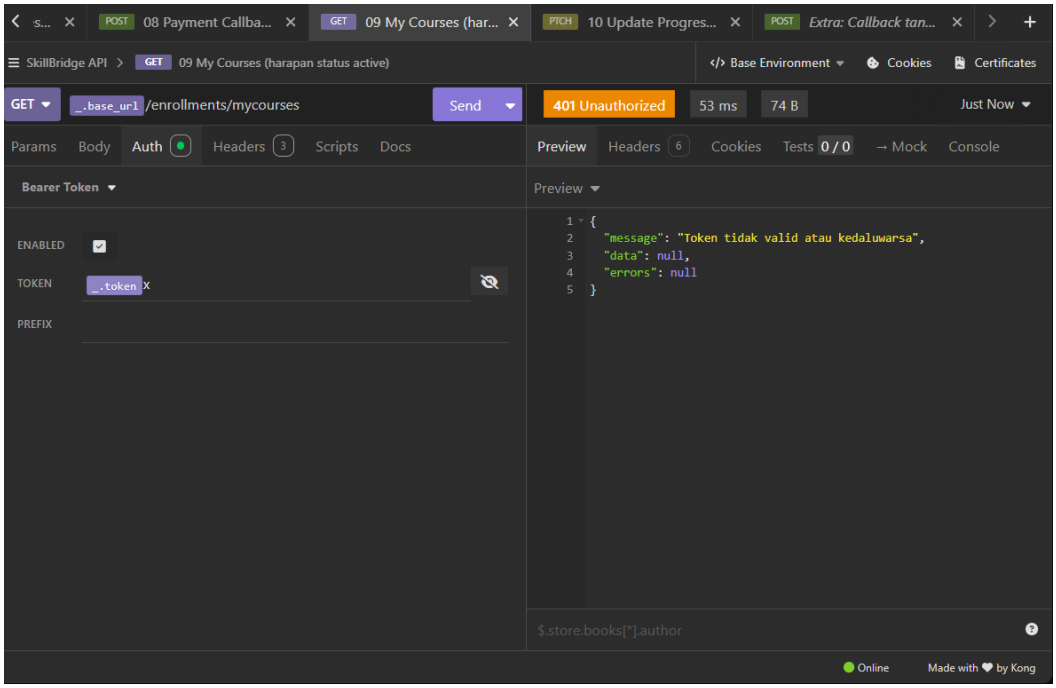
Pilih request 09 My Courses atau request apa pun yang memerlukan autentikasi. Pada tab Auth, pastikan statusnya dikosongkan (No Auth). Kemudian klik tombol Send



Pengujian 2: Menggunakan Token yang Tidak Valid (Ngawur)

Langkah Pengujian:

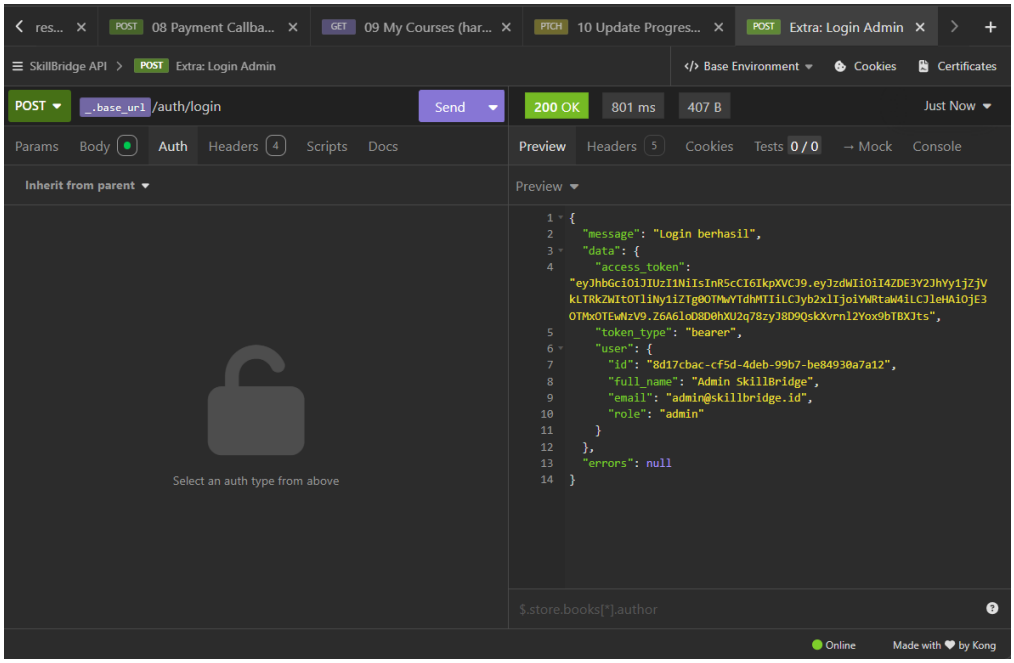
Pilih request yang membutuhkan autentikasi (misal: 09 My Courses). Pada tab **Auth**, ubah atau tambahkan satu karakter acak pada string token Anda (misal: tambahkan huruf x di ujung token). Klik tombol **Send**.

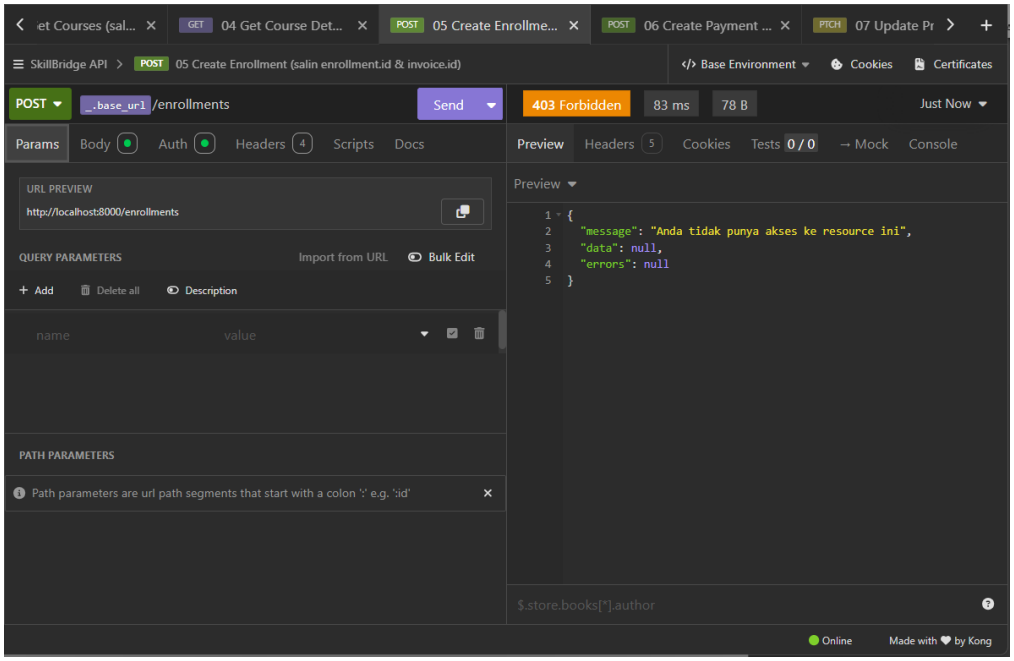


Pengujian 3: Salah Role (Admin Mencoba Melakukan Enrollment)

Langkah Pengujian:

Login terlebih dahulu sebagai admin menggunakan request Extra: Login Admin (admin@skillbridge.id) dan salin token *access_token*-nya ke variabel environment token. Buka request 05 Create Enrollment (yang seharusnya hanya boleh diakses oleh *student*). Klik tombol **Send**.





Pengujian 4: Student Mencoba Membuat Kursus Baru

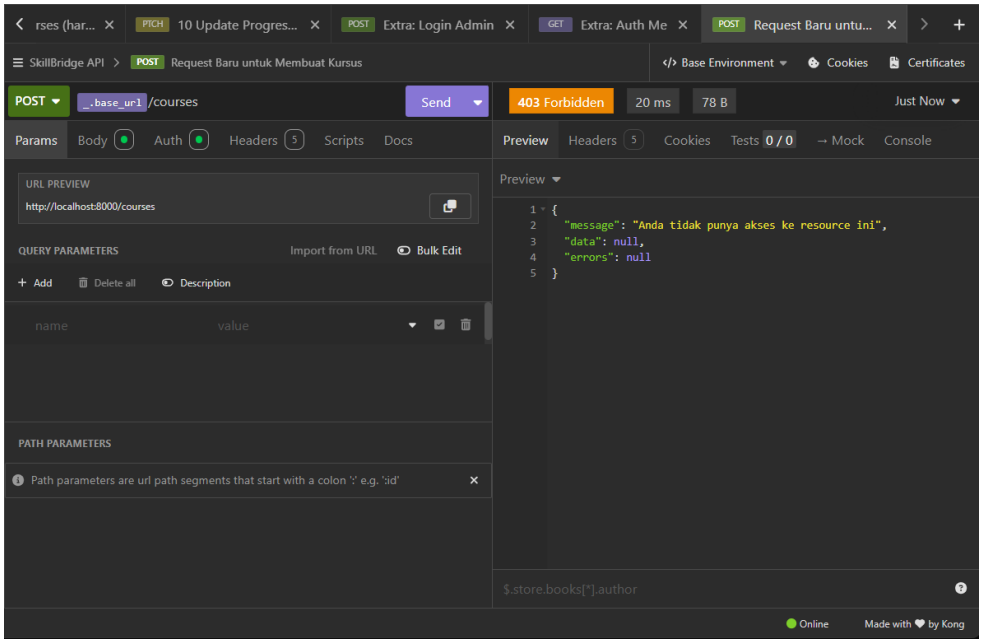
Langkah Pengujian:

Di aplikasi Insomnia, buat sebuah *Request* baru dengan ketentuan berikut:

Method	POST
URL	{{ _.base_url }}/courses

Authentication	Pilih **Bearer Token**, lalu masukkan {{ _.token }}
Headers	Tambahkan Content-Type: application/json

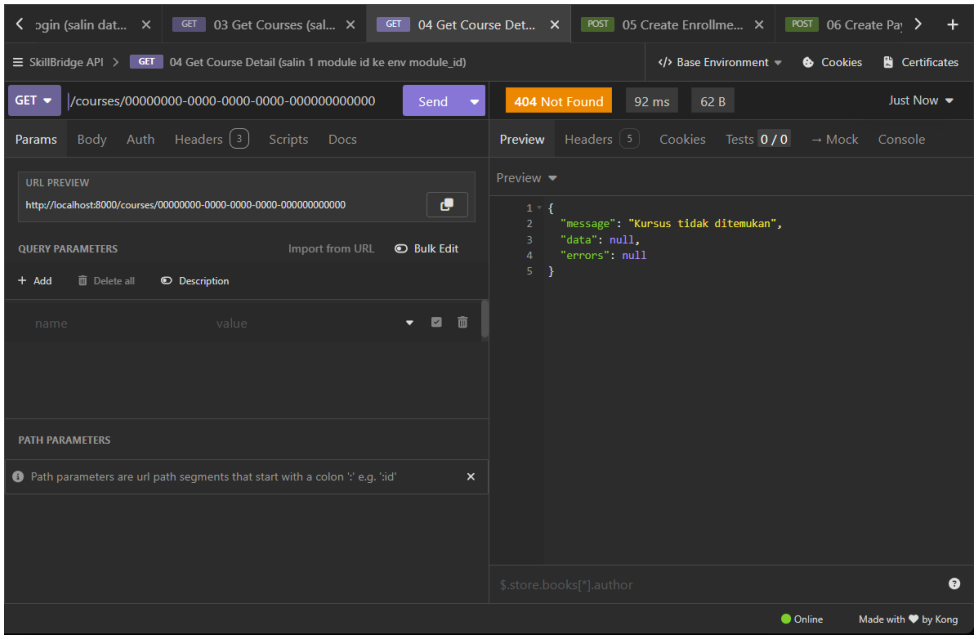
Body (Body JSON)	{
	"category_id": "00000000-0000-0000-0000-000000000000",
	"title": "Kursus Ilegal oleh Student",
	"description": "Mencoba membuat kursus tanpa hak akses",
	"price": "100000.00"
	}



Pengujian 5: Pencarian ID Kursus yang Tidak Ada di Database

Langkah Pengujian:

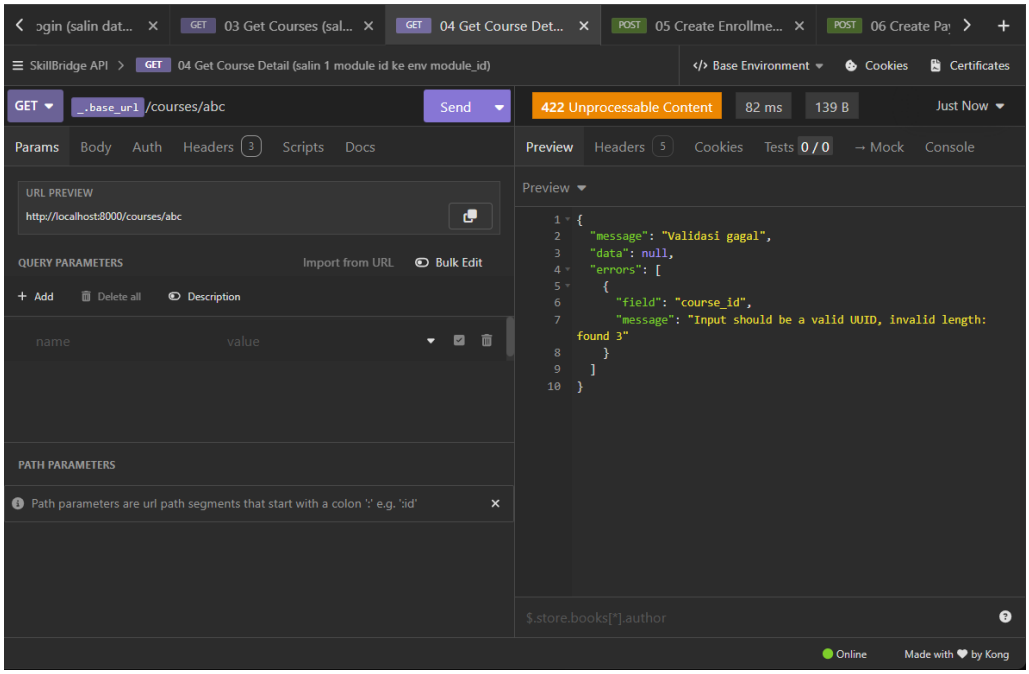
Gunakan request yang sudah ada untuk melihat detail kursus (biasanya bernama 04 Get Course Detail atau mirip). Masukkan URL berikut pada kolom alamat request: {{ _.base_url }}/courses/00000000-0000-0000-0000-000000000000



Pengujian 6: Format ID Kursus Salah / Bukan UUID

Langkah Pengujian:

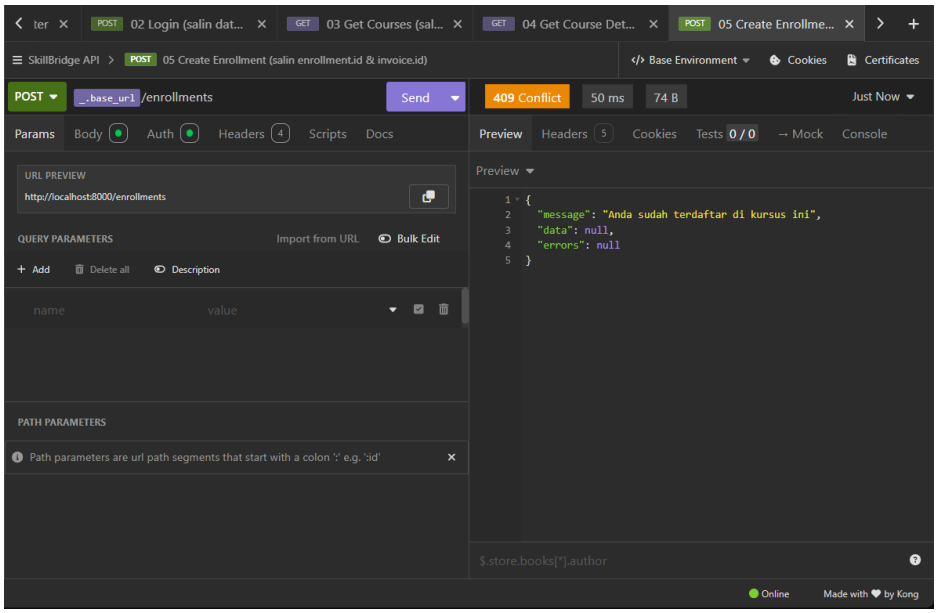
Gunakan request *Get Course Detail* di Insomnia. Ubah URL endpoint dengan memasukkan string acak yang **bukan format UUID** (misal: /courses/abc atau /courses/12345). Klik tombol **Send**.



Pengujian 7: Pendaftaran Kursus Dua Kali / *Enroll* Ganda

Langkah Pengujian:

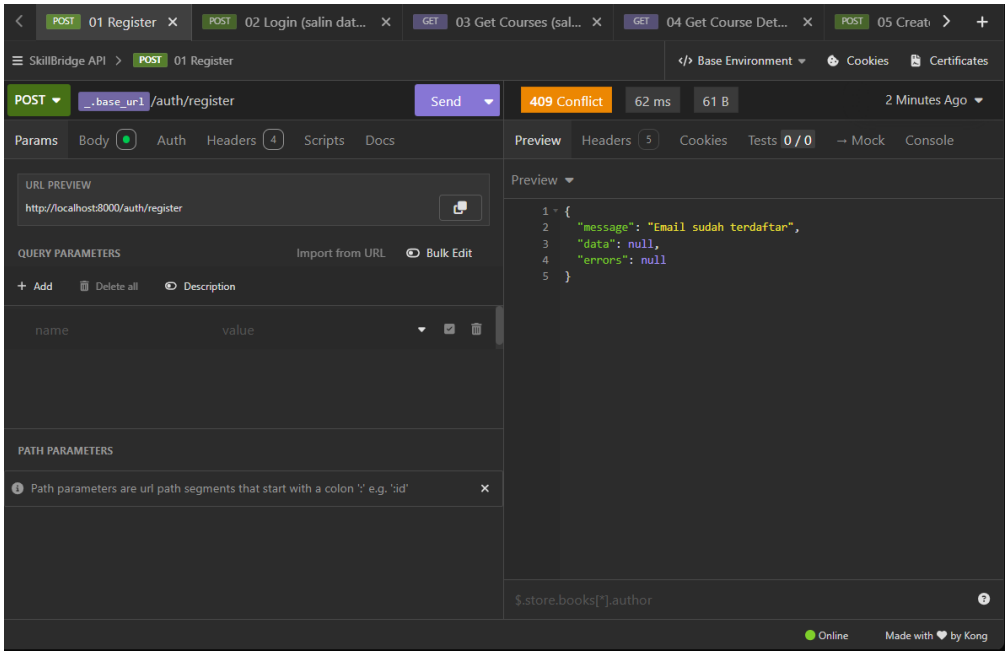
Gunakan kembali request 05 Create Enrollment dengan *Bearer Token* akun *student* yang sama dan course_id kursus yang sama. Pastikan akun tersebut sebelumnya **sudah pernah** berhasil mendaftar ke kursus tersebut. Klik tombol **Send** untuk kedua kalinya.



Pengujian 8: Registrasi dengan Email yang Sudah Terdaftar

Langkah Pengujian:

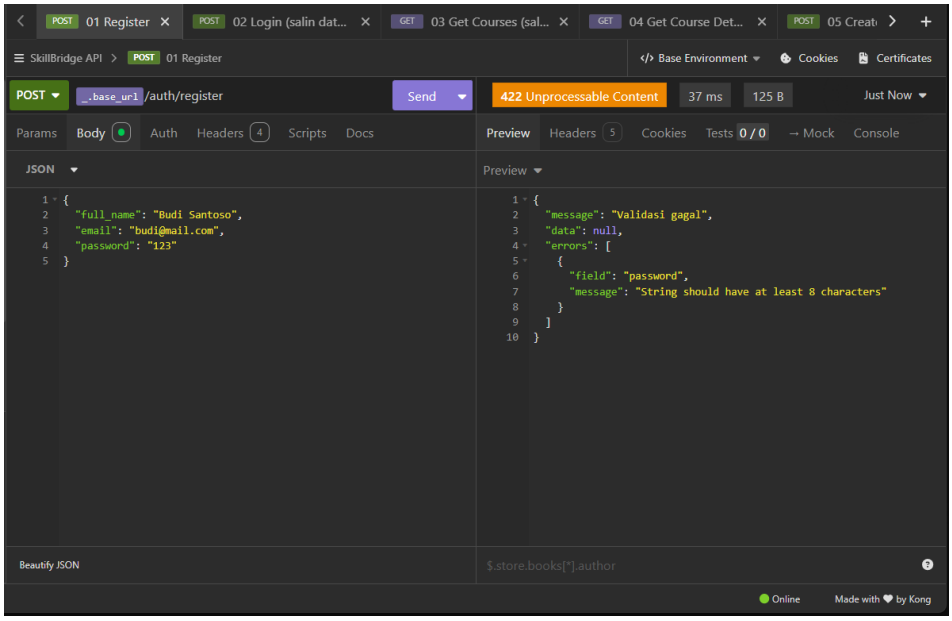
Buka request 01 Register. Masukkan data registrasi menggunakan **email yang sama persis** dengan akun yang sudah pernah didaftarkan sebelumnya (misal: budi@mail.com). Klik tombol **Send.**



Pengujian 9: Password Pendek

Langkah Pengujian:

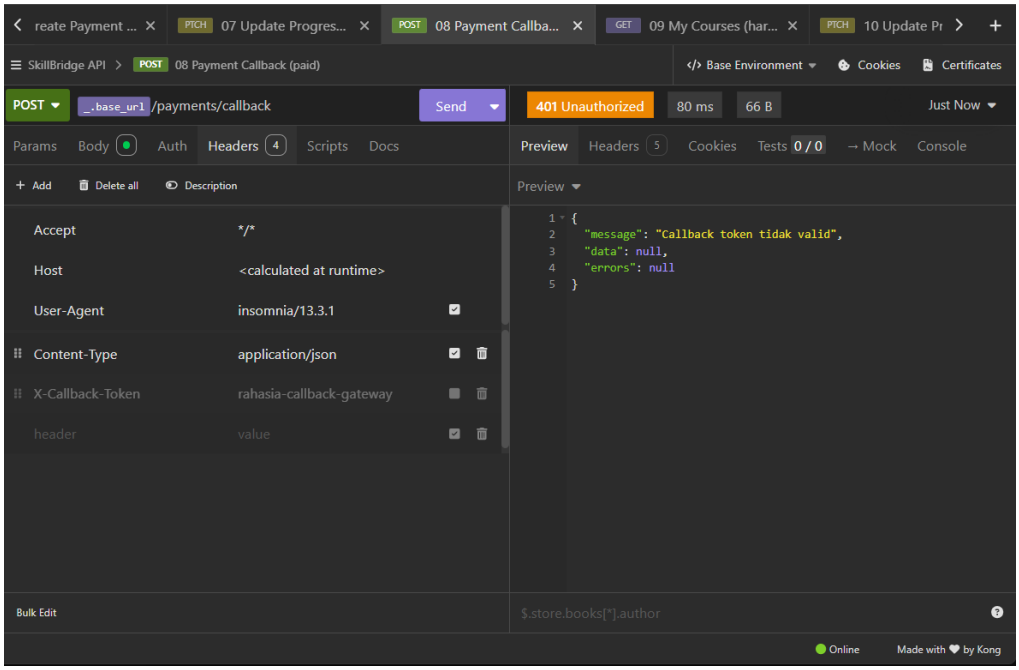
Buka request **01 Register** di dalam *collection* Insomnia.Ubah isi *Body* JSON dengan memasukkan password yang kurang dari 8 karakter



Pengujian 10: Callback Pembayaran Tanpa Header Token (*Unauthorized*)

Langkah Pengujian:

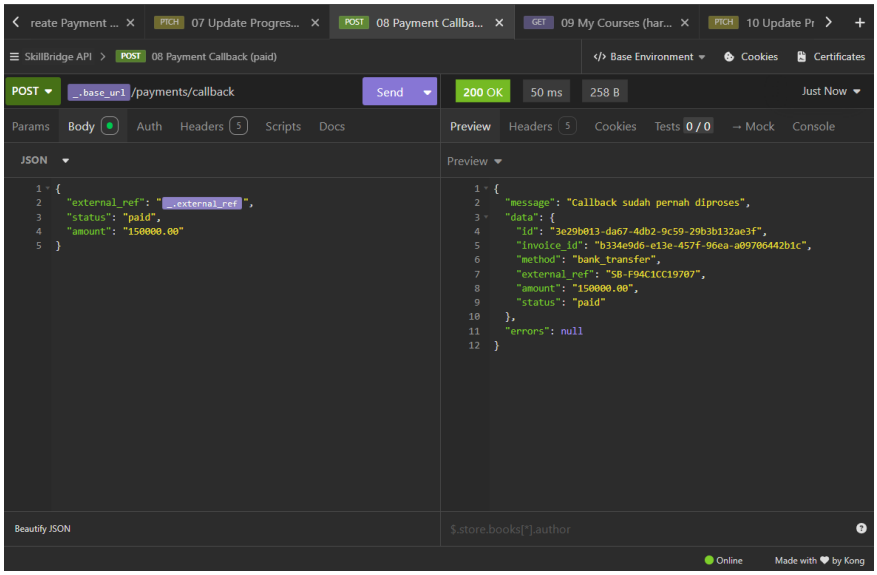
Buka request *Payment Callback* (POST /payments/callback). Pada tab **Headers**, **hapus** header X-Callback-Token (atau kosongkan nilainya). Masukkan body JSON pembayaran yang valid, lalu klik tombol **Send**.



Pengujian 11: Callback Dua kali

Langkah Pengujian:

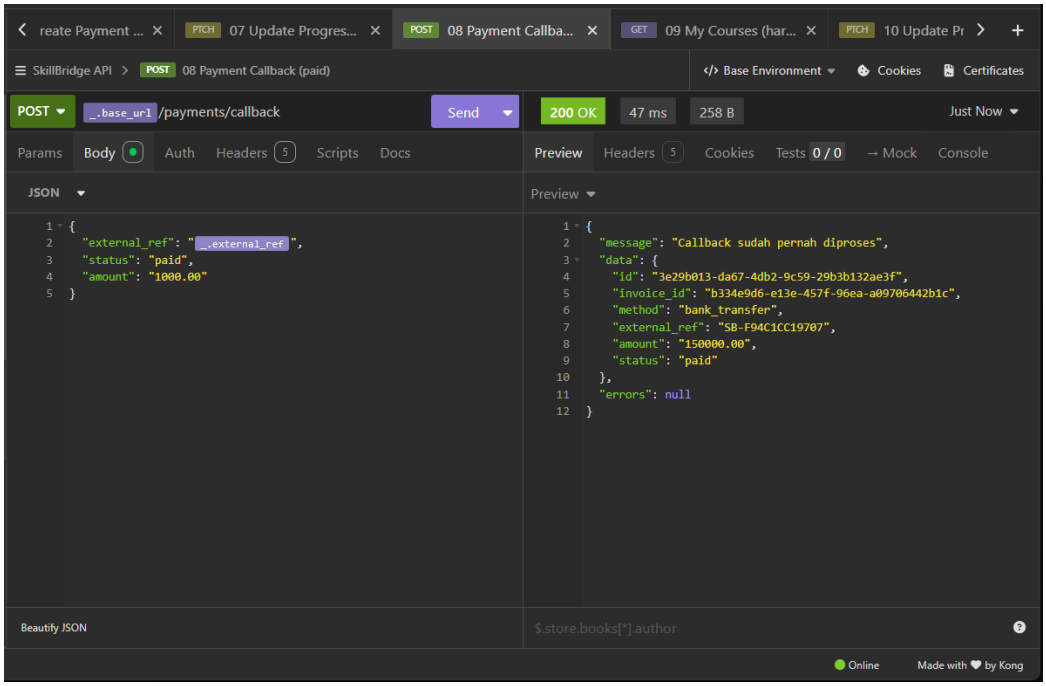
Pastikan Anda sudah menjalankan alur hingga tahap sukses melakukan *callback* pembayaran (Langkah 8 di Insomnia yang menghasilkan status 200 OK pertama kalinya). Tetap berada di request **08 Payment Callback (paid)** dengan body JSON dan header X-Callback-Token (rahasia-callback-gateway) yang sama persis seperti sebelumnya. Klik tombol **Send** untuk **kedua kalinya** (mengirim data *callback* dengan external_ref yang sama).



Pengujian 12: Callback Pembayaran dengan Nominal Berbeda

Langkah Pengujian:

Buka request *Payment Callback* (POST /payments/callback). Pastikan header X-Callback-Token sudah terisi dengan rahasia-callback-gateway. Pada tab **Body**, ubah nilai "amount" menjadi nominal yang **tidak sesuai** dengan tagihan asli (misal tagihan asli 150000.00, diisi 1000.00). Klik tombol **Send**.



No	Skenario	Hasil Diharapkan	Hasil Aktual	Keterangan
1.	Tanpa Token (Get My Courses)	401	401	Berhasil
2.	Token Ngawur	401	401	Berhasil
3.	Salah Role	403	403	Berhasil
4.	Student buat Kursus	403	403	Berhasil
5.	ID Kursus Tidak Ada	404	404	Berhasil
6.	ID Kursus Bukan UUID	422	422	Berhasil
7.	Enroll Dua Kali	409	409	Berhasil
8.	Email Double	409	409	Berhasil
9.	Password Pendek	422 + Array Errors	422	Berhasil
10.	Callback Pembayaran Tanpa Header Token	401	401	Berhasil
11.	Callback Dua Kali	200	200	Berhasil
12.	Callback Pembayaran dengan Nominal Berbeda	422	200	Gagal

D. Panduan Verifikasi Database PostgreSQL Lokal

Buka terminal PowerShell, masuk ke database menggunakan user skillbridge:

psql -U skillbridge -d skillbridge

(Masukkan password secret saat diminta). Jalankan query berikut untuk memverifikasi integritas data:

```
C:\Windows\System32>psql -U skillbridge -d skillbridge
Password for user skillbridge:

psql (18.6)
WARNING: Console code page (850) differs from Windows code page (1252)
         8-bit characters might not work correctly. See psql reference
         page "Notes for Windows users" for details.
Type "help" for help.
```

Verifikasi Hash Password: SELECT email, role, left(password_hash, 7) AS hash_prefix FROM users;

```
skillbridge=> SELECT email, role, left(password_hash, 7) AS hash_prefix FROM users;
          email           |    role    | hash_prefix
--------------------------+------------+-------------
 admin@skillbridge.id     | admin      | $2b$12$
 instructor@skillbridge.id | instructor | $2b$12$
 budi@mail.com            | student    | $2b$12$
(3 rows)
```

Verifikasi Relasi Order & Invoice: SELECT o.id AS order_id, o.status AS order_status, i.invoice_number, i.status AS invoice_status

FROM orders o JOIN invoices i ON i.order_id = o.id;

```
skillbridge=> SELECT o.id AS order_id, o.status AS order_status, i.invoice_number, i.status AS invoice_status
skillbridge-> FROM orders o JOIN invoices i ON i.order_id = o.id;
              order_id              | order_status |    invoice_number     | invoice_status
------------------------------------+--------------+-----------------------+----------------
 403dca58-559f-4e46-a38c-067e48b91b17 | paid         | INV-20261028-403DCA58 | paid
(1 row)
```

Verifikasi Sinkronisasi Pembayaran & Enrollment:

SELECT p.status AS payment, i.status AS invoice, o.status AS "order", e.status AS enrollment

FROM payments p

JOIN invoices i ON i.id = p.invoice_id

JOIN orders o ON o.id = i.order_id

JOIN enrollments e ON e.order_id = o.id;

```
skillbridge=> SELECT p.status AS payment, i.status AS invoice, o.status AS "order", e.status AS enrollment
skillbridge-> FROM payments p
skillbridge-> JOIN invoices i ON i.id = p.invoice_id
skillbridge-> JOIN orders o ON o.id = i.order_id
skillbridge-> JOIN enrollments e ON e.order_id = o.id;
 payment | invoice | order | enrollment
---------+---------+-------+------------
 paid    | paid    | paid  | active
(1 row)
```

Verifikasi Constraint Database: UPDATE enrollments SET status = 'ngawur';

```
skillbridge=> UPDATE enrollments SET status = 'ngawur';
ERROR:  new row for relation "enrollments" violates check constraint "ck_enrollments_status"
DETAIL:  Failing row contains (e9a168c0-1525-42a1-944b-5910bb3e077c, 9ff22cee-6fd4-4bae-8119-97eaa5ad2784, d4859ef2-cd7c
-403a-ac29-dccb4c7bdd5a, 403dca58-559f-4e46-a38c-067e48b91b17, ngawur, 2026-10-28 18:36:12.502133+08).
skillbridge=>
```